ASSIGNMENT REPORT

OnlineFoodOrderManagementandAnalysis System

Submitted by: JACK YANNIS F | **Register Number:** 1924260159

1. ABSTRACT

This project presents the design and implementation of an Online Food Order Management and Analysis System. Developed in Python, the system provides a menu-driven interface to manage customers, restaurants, food items, and orders efficiently. By leveraging fundamental Python data structures like dictionaries, lists, sets, and tuples, alongside file handling and exception management, the system offers a robust simulation of a real-world food delivery backend. The objective is to automate workflows, calculate bills dynamically, and generate comprehensive analytical reports.

2. INTRODUCTION

In the modern era of digital convenience, food delivery platforms have become indispensable. Managing the complex interactions between multiple restaurants, diverse customer preferences, and dynamic order tracking requires a reliable software infrastructure. This project aims to build a prototype system that handles these operations seamlessly. It demonstrates the application of Python programming concepts including control structures, file I/O, and custom modular functions to solve real-world logistical challenges in the food service industr y.

3. PROBLEM STATEMENT

Design, implement, and evaluate a menu-driven, Python-based Online Food Order Management and Analysis System for a large food-delivery platform, where multiple restaurants, food items, customers, and delivery orders must be managed efficiently and manual coordination has become unreliable. The prototype shall automate the food-ordering workflow registering customers, managing restaurants and food items, creating and updating orders, calculating order totals, applying discounts and delivery charges, tracking order status, comparing orders, and generating an order and restaurant-performance report. The core entities (customers, restaurants, food items, and orders) shall be modelled using suitable Python data structures such as dictionaries, lists, tuples, and sets, along with functions and control structures. Different order statuses and food categories shall be handled through appropriate conditional and iterative logic, so order processing, billing, searching, and analysis can adapt dynamically at run time. Robustness shall be ensured through functions, modules, exception handling, file handling, input validation, and at least two meaningful custom operations e.g., comparing orders based on total amount and generating formatted order-report output. The system shall demonstrate efficient data storage, retrieval, string manipulation, analysis, and persistence of customer and order information using Python.

4. PSEUDO CODE

```
START
INITIALIZE empty dictionaries for Customers, Restaurants, Orders
SET order_id_counter = 100

FUNCTION display_menu():
```

```
    PRINT "1. Register Customer"
    PRINT "2. Add Restaurant & Menu"
    PRINT "3. Place Order"
    PRINT "4. View Orders & Generate Report"
    PRINT "5. Exit"

WHILE True:
    CALL display_menu()
    READ choice
    IF choice == 1:
        READ customer_id, name
        Customers[customer_id] = name
        PRINT "Customer Registered"
    ELSE IF choice == 2:
        READ rest_id, rest_name, item_details
        Restaurants[rest_id] = {rest_name, menu: item_details}
        PRINT "Restaurant Added"
    ELSE IF choice == 3:
        READ customer_id, rest_id, items_ordered
        TRY:
            VALIDATE customer_id and rest_id
            CALCULATE base_total = sum(prices of items_ordered)
            CALCULATE final_total = base_total + delivery_charge
            Orders[order_id_counter] = {customer_id, rest_id, items, final_total, status:
"Pending"}
            INCREMENT order_id_counter
            SAVE to file "orders_data.txt"
            PRINT "Order Placed Successfully"
        EXCEPT KeyError:
            PRINT "Invalid ID provided"
    ELSE IF choice == 4:
        READ data from "orders_data.txt"
        GENERATE formatted report of all orders
        COMPARE orders to find highest value order
    ELSE IF choice == 5:
        BREAK
    ELSE:
        PRINT "Invalid Choice"
END
```

5. IMPLEMENTATION CODE

```
import json
import os

# Global Data Structures
customers = {}
restaurants = {}
orders = {}
order_counter = 100

def load_data():
    global order_counter
    if os.path.exists("orders_data.txt"):
        try:
            with open("orders_data.txt", "r") as f:
                data = json.load(f)
                return data.get("orders", {}), data.get("counter", 100)
        except Exception as e:
            print("Error loading data:", e)
            return {}, 100
```

```
    return {}, 100

def save_data():
    with open("orders_data.txt", "w") as f:
        json.dump({"orders": orders, "counter": order_counter}, f)

def register_customer():
    c_id = input("Enter Customer ID: ")
    name = input("Enter Customer Name: ")
    customers[c_id] = name
    print(f"Customer {name} registered successfully!\n")

def add_restaurant():
    r_id = input("Enter Restaurant ID: ")
    r_name = input("Enter Restaurant Name: ")
    restaurants[r_id] = {"name": r_name, "menu": {}}
    while True:
        item = input("Enter Food Item (or 'done' to stop): ")
        if item.lower() == 'done':
            break
        try:
            price = float(input(f"Enter price for {item}: "))
            restaurants[r_id]["menu"][item] = price
        except ValueError:
            print("Invalid price! Try again.")
    print(f"Restaurant {r_name} added!\n")

def place_order():
    global order_counter
    try:
        c_id = input("Enter Customer ID: ")
        if c_id not in customers:
            raise KeyError("Customer not found!")
        r_id = input("Enter Restaurant ID: ")
        if r_id not in restaurants:
            raise KeyError("Restaurant not found!")

        print("Menu:", restaurants[r_id]["menu"])
        ordered_items = []
        total_amount = 0

        while True:
            item = input("Enter item to order (or 'done' to finish): ")
            if item.lower() == 'done':
                break
            if item in restaurants[r_id]["menu"]:
                ordered_items.append(item)
                total_amount += restaurants[r_id]["menu"][item]
            else:
                print("Item not on menu!")

        if not ordered_items:
            print("No items ordered.")
            return

        final_amount = total_amount + 50 # Rs 50 Flat delivery charge
        order_id = f"ORD{order_counter}"
        orders[order_id] = {
            "customer": customers[c_id],
            "restaurant": restaurants[r_id]["name"],
            "items": ordered_items,
            "total": final_amount,
            "status": "Confirmed"
```

```
            }
            order_counter += 1
            save_data()
            print(f"Order {order_id} placed! Total Amount: Rs.{final_amount}\n")
        except KeyError as e:
            print(f"Error: {e}\n")
        except Exception as e:
            print(f"Unexpected Error: {e}\n")

 def generate_report():
        if not orders:
            print("No orders to display.")
            return
        print("-" * 50)
        print("ORDER REPORT")
        print("-" * 50)
        highest_order = None
        max_amt = 0
        for oid, details in orders.items():
            print(f"[{oid}] {details['customer']} ordered from {details['restaurant']}")
            print(f"Items: {', '.join(details['items'])} | Total: Rs.{details['total']}")
            print(f"Status: {details['status']}")
            print("-" * 30)
            if details['total'] > max_amt:
            max_amt = details['total']
            highest_order = oid
        print(f"Highest Value Order: {highest_order} (Rs.{max_amt})")
        print("-" * 50 + "\n")

def main():
        global orders, order_counter
        orders, order_counter = load_data()
        while True:
            print("=== FOOD ORDER MANAGEMENT SYSTEM ===")
            print("1. Register Customer")
            print("2. Add Restaurant & Menu")
            print("3. Place Order")
            print("4. Generate Report")
            print("5. Exit")
            choice = input("Enter choice: ")
            if choice == '1':
                register_customer()
                 elif choice == '2':
                 add_restaurant()
                elif choice == '3':
                place_order()
            elif choice == '4':
                generate_report()
            elif choice == '5':
                print("Exiting system. Have a great day!")
                break
            else:
                print("Invalid choice. Please select 1-5.\n")

if __name__ == "__main__":
        main()
```

6. OUTPUT SCREENS (SIMULATED)

```
=== FOOD ORDER MANAGEMENT SYSTEM ===
1. Register Customer
```

```
2. Add Restaurant & Menu
3. Place Order
4. Generate Report
5. Exit
Enter choice: 1
Enter Customer ID: C01
Enter Customer Name: A.RITHIN SAI
Customer A.RITHIN SAI registered successfully!


Enter choice: 2
Enter Restaurant ID: R01
Enter Restaurant Name: Spice Villa
Enter Food Item (or 'done' to stop): Paneer Tikka
Enter price for Paneer Tikka: 200
Enter Food Item (or 'done' to stop): done
Restaurant Spice Villa added!


Enter choice: 3
Enter Customer ID: C01
Enter Restaurant ID: R01
Menu: {'Paneer Tikka': 200.0}
Enter item to order (or 'done' to finish): Paneer Tikka
Enter item to order (or 'done' to finish): done
Order ORD100 placed! Total Amount: Rs.250.0


Enter choice: 4
------------------------------------------------
ORDER REPORT
------------------------------------------------
[ORD100] A.RITHIN SAI ordered from Spice Villa
Items: Paneer Tikka | Total: Rs.250.0
Status: Confirmed
----------------------------
Highest Value Order: ORD100 (Rs.250.0)
------------------------------------------------
```

7. CONCLUSION

The Online Food Order Management and Analysis System successfully demonstrates the application of Python programming to automate and streamline digital food ordering processes. By implementing modular functions, structured data storage (dictionaries and lists), exception handling, and file I/O operations, the system achieves robustness and adaptability. The assignment reinforced essential computer science concepts, specifically in dynamic data manipulation and algorithm design, fulfilling all the requirements outlined in the problem statement.